Practical 07

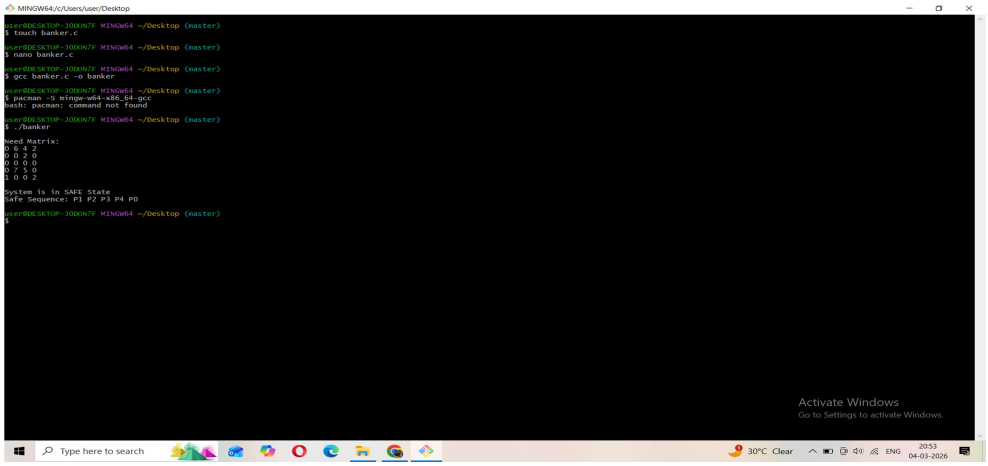
Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Code:

```
#include <stdio.h>

int main() {
   int n = 5;   // Number of processes
   int r = 4;   // Number of resources

   int allocation[5][4] = {
      {0,0,1,4},
      {0,6,3,2},
      {0,0,1,2},
      {1,0,0,0},
      {1,3,5,4}
   };

   int max[5][4] = {
      {0,6,5,6},
      {0,6,5,2},
```

```
    {0,0,1,2},
    {1,7,5,0},
    {2,3,5,6}
};

int available[4] = {1,6,2,0};

int need[5][4];
int finish[5] = {0};
int safeSeq[5];
int work[4];

// Calculate Need Matrix
for(int i=0;i<n;i++){
    for(int j=0;j<r;j++){
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

printf("\nNeed Matrix:\n");
for(int i=0;i<n;i++){
    for(int j=0;j<r;j++){
        printf("%d ", need[i][j]);
    }
    printf("\n");
}
```

```
// Initialize work = available
for(int i=0;i<r;i++)
    work[i] = available[i];

int count = 0;

while(count < n) {
    int found = 0;

    for(int i=0;i<n;i++) {
        if(finish[i] == 0) {

            int possible = 1;

            for(int j=0;j<r;j++) {
                if(need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }

            if(possible) {
                for(int j=0;j<r;j++)
                    work[j] += allocation[i][j];

                safeSeq[count++] = i;
                finish[i] = 1;
```

```
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("\nSystem is NOT in Safe State\n");
        return 0;
    }
  }

  printf("\nSystem is in SAFE State\nSafe Sequence: ");
  for(int i=0;i<n;i++)
    printf("P%d ", safeSeq[i]);

  printf("\n");
  return 0;
}
```

Output